

Assignment: Hand class

Introduction to Software Engineering CSCI 2360

Objective

Learn how to write more Python by adding functionality to the existing `Deck` class.

Overview

You will modify the `Deck` class in `deck.py` to implement methods needed for the deck class. This assignment will help you understand:

- The `Deck` class implementation.
- How the `Deck` class uses the `Card` class and the `Hand` class.

Background:

Pitch has a deck of cards, which is represented in `deck.py`.

Task Description

The `Deck` class already exists in `deck.py` with some functionality. The `Card` class and `Hand` class are also provided. Your task is to implement

- `shuffle`
- `draw`
- `deal`

Method 1: `shuffle`

Implement a method that shuffles the deck.

Method Signature:

```
def shuffle(self):  
    """  
    Randomly reorders all cards currently in the deck.  
  
    Performs an in-place Fisher-Yates shuffle of  
    the cards list. This ensures each possible ordering has equal  
probability  
    and provides fair randomization for card games.  
  
    Note: Only shuffles cards currently in the deck. If cards have  
been  
    drawn or dealt, those cards are not affected.  
    """
```

Method 2: draw

Implement a method that draws a card from the deck.

Method Signature:

```
def draw(self):
    """
    Removes and returns the last card from the deck.

    Removes the card at the end of the cards list,
    effectively drawing from the "top" of the deck. The drawn card is
    permanently removed from the deck until reset() is called.

    Returns:
        Card or None: The drawn Card object if the deck is not empty,
        or None if the deck is empty (no cards remaining).
    """
```

Method 2: deal

Implement a method that deals cards from the deck.

Method Signature:

```
def deal(self, nhands=4, ncards=9):
    """
    Randomly distributes cards from the deck into multiple hands.

    Uses random sampling to fairly distribute cards among players,
    ensuring
    no card appears in multiple hands. Each hand is automatically
    sorted
    by rank in descending order after dealing. Cards are permanently
    removed from the deck once dealt.

    Args:
        nhands (int, optional): The number of hands to create.
        Defaults to 4
        for a standard 4-player Pitch game.
        ncards (int, optional): The number of cards per hand. Defaults
        to 9
        for standard Pitch dealing. If 0, creates empty hands.

    Returns:
        list[list[Card]]: A list of hands, where each hand is a list
        of Card
        objects sorted by rank in descending order. Returns empty
        hands
        if ncards is 0.
    """
```

```
Raises ValueError if trying to deal
more cards than remain in the deck.
.....
```

Available Class Attributes and Methods:

- `self.cards`: (list) List of Card objects in the hand.

Example Usage

See `main` in `deck.py`.

Grading Rubric

See Autograder results for distribution of points.

Submission Requirements

1. Complete the implementation of methods in `deck.py`.
2. Test your implementation with the provided test cases (use `python -m`).
3. Ensure your code follows Python style conventions (consider using `pylint`).
4. Push your modified `deck.py` file to GitHub.

Grading Rubric

See Autograder results for distribution of points.

Due Dates

The due date is specified on Blackboard.

Good luck! Remember to test your code thoroughly and ask questions if you need clarification.

© Copyright 2025 by Michelle Talley

You may not publish this document on any website or share it with anyone without explicit permission of the author.